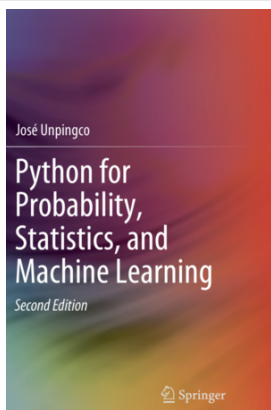


```
from IPython.display import Image
Image('../Python_probability_statistics_machine_learning_2E.png',width=200)
```



✓ Monte Carlo Sampling Methods

So far, we have studied analytical ways to transform random variables and how to augment these methods using Python. In spite of all this, we frequently must resort to purely numerical methods to solve real-world problems. Hopefully, now that we have seen the deeper theory, these numerical methods will feel more concrete. Suppose we want to generate samples of a given density, $f(x)$, given we already can generate samples from a uniform distribution, $\mathcal{U}[0, 1]$. How do we know a random sample v comes from the $f(x)$ distribution? One approach is to look at how a histogram of samples of v approximates $f(x)$. Specifically,

$$\mathbb{P}(v \in N_{\Delta}(x)) = f(x)\Delta x \quad (1)$$

The histogram approximates the target probability density.



which says that the probability that a sample is in some N_{Δ} neighborhood of x is approximately $f(x)\Delta x$. [Figure](#) shows the target probability density function $f(x)$ and a histogram that approximates it. The histogram is generated from samples v . The hatched rectangle in the center illustrates Equation 1. The area of this rectangle is approximately $f(x)\Delta x$ where $x = 0$, in this case. The width of the rectangle is $N_{\Delta}(x)$. The quality of the approximation may be clear visually, but to know that v samples are characterized by $f(x)$, we need the statement of Equation 1, which says that the proportion of samples v that fill the hatched rectangle is approximately equal to $f(x)\Delta x$.

Now that we know how to evaluate samples v that are characterized by the density $f(x)$, let's consider how to create these samples for both discrete and continuous random variables.

Inverse CDF Method for

Discrete Variables

Suppose we want to generate samples from a fair six-sided die. Our workhouse uniform random variable is defined continuously over the unit interval and the fair six-sided die is discrete. We must first create a mapping between the continuous random variable u and the discrete outcomes of the die. This mapping is shown in [Figure](#) where the unit interval is broken up into segments, each of length $1/6$. Each individual segment is assigned to one of the die outcomes. For example, if $u \in [1/6, 2/6)$, then the outcome for the die is 2. Because the die is fair, all segments on the unit interval are the same length. Thus, our new random variable v is derived from u by this assignment.

A uniform distribution random variable on the unit interval is assigned to the six outcomes of a fair die using these segments.



For example, for $v = 2$, we have,

$$\mathbb{P}(v = 2) = \mathbb{P}(u \in [1/6, 2/6)) = 1/6$$

where, in the language of the Equation 1, $f(x) = 1$ (uniform distribution), $\Delta x = 1/6$, and $N_{\Delta}(2) = [1/6, 2/6)$. Naturally, this pattern holds for all the other die outcomes in $\{1, 2, 3, \dots, 6\}$. Let's consider a quick simulation to make this concrete. The following code generates uniform random samples and stacks them in a Pandas dataframe.

```
import pandas as pd
import numpy as np
from pandas import DataFrame
```

```
u= np.random.rand(100)
df = DataFrame(data=u,columns=['u'])
```

The next block uses `pd.cut` to map the individual samples to the set $\{1, 2, \dots, 6\}$ labeled `v`.

```
labels = [1,2,3,4,5,6]
df['v']=pd.cut(df.u,np.linspace(0,1,7),
               include_lowest=True,labels=labels)
```

This is what the dataframe contains. The `v` column contains the samples drawn from the fair die.

```
df.head()
```

	u	v
0	0.431806	3
1	0.810623	5
2	0.451704	3
3	0.028535	1
4	0.703825	5

Các bước tiếp theo: [Tạo mã bằng df](#) [New interactive sheet](#)

The following is a count of the number of samples in each group. There should be roughly the same number of samples in each group because the die is fair.

```
df.groupby('v').count()
```

```
/tmp/ipykernel_4050/2710018620.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a
df.groupby('v').count()
```

	u
v	
1	20
2	11
3	18
4	19
5	23
6	9

So far, so good. We now have a way to simulate a fair die from a uniformly distributed random variable.

To extend this to unfair die, we need only make some small adjustments to this code. For example, suppose that we want an unfair die so that $\mathbb{P}(1) = \mathbb{P}(2) = \mathbb{P}(3) = 1/12$ and $\mathbb{P}(4) = \mathbb{P}(5) = \mathbb{P}(6) = 1/4$. The only change we have to make is with `pd.cut` as follows,

```
df['v']=pd.cut(df.u,[0,1/12,2/12,3/12,2/4,3/4,1],
               include_lowest=True,labels=labels)
df.groupby('v').count()/df.shape[0]
```

```
/tmp/ipykernel_4050/1018620274.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to True in a
df.groupby('v').count()/df.shape[0]
```

```
u  

v
1  0.07
2  0.13
3  0.06
4  0.23
5  0.30
6  0.21
```

where now these are the individual probabilities of each digit. You can take more than `100` samples to get a clearer view of the individual probabilities but the mechanism for generating them is the same. The method is called the inverse CDF [[^]CDF] method because the CDF (namely, `[0, 1/12, 2/12, 3/12, 2/4, 3/4, 1]`) in the last example has been inverted (using the `pd.cut` method) to generate the samples.

The inversion is easier to see for continuous variables, which we consider next.

[[^]CDF]: Cumulative density function. Namely, $F(x) = \mathbb{P}(X < x)$.

✓ Inverse CDF

Method for Continuous Variables

The method above applies to continuous random variables, but now we have to squeeze the intervals down to individual points. In the example above, our inverse function was a piecewise function that operated on uniform random samples. In this case, the piecewise function collapses to a continuous inverse function. We want to generate random samples for a CDF that is invertible. As before, the criterion for generating an appropriate sample v is the following,

$$\mathbb{P}(F(x) < v < F(x + \Delta x)) = F(x + \Delta x) - F(x) = \int_x^{x+\Delta x} f(u) du \approx f(x) \Delta x$$

which says that the probability that the sample v is contained in a Δx interval is approximately equal to $f(x) \Delta x$, at that point. Once again, the trick is to use a uniform random sample u and an invertible CDF $F(x)$ to construct these samples. Note that for a uniform random variable $u \sim \mathcal{U}[0, 1]$, we have,

$$\begin{aligned} \mathbb{P}(x < F^{-1}(u) < x + \Delta x) &= \mathbb{P}(F(x) < u < F(x + \Delta x)) \\ &= F(x + \Delta x) - F(x) \\ &= \int_x^{x+\Delta x} f(p) dp \approx f(x) \Delta x \end{aligned}$$

This means that $v = F^{-1}(u)$ is distributed according to $f(x)$, which is what we want.

Let's try this to generate samples from the exponential distribution,

$$f_{\alpha}(x) = \alpha e^{-\alpha x}$$

which has the following CDF,

$$F(x) = 1 - e^{-\alpha x}$$

and corresponding inverse,

$$F^{-1}(u) = \frac{1}{\alpha} \ln \frac{1}{(1-u)}$$

Now, all we have to do is generate some uniformly distributed random samples and then feed them into F^{-1} .

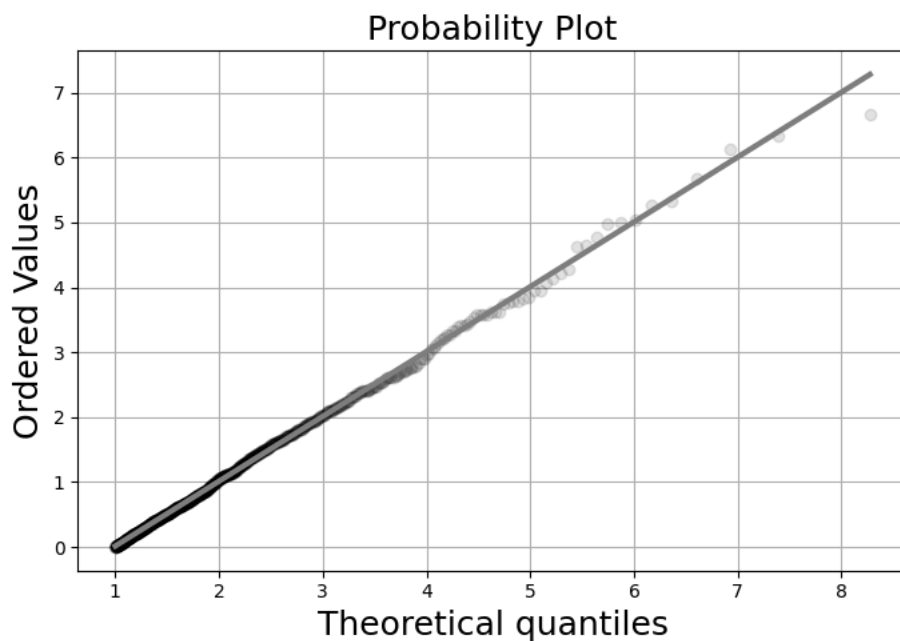
```
from numpy import array, log
import scipy.stats
alpha = 1. # distribution parameter
nsamp = 1000 # num of samples
# define uniform random variable
u=scipy.stats.uniform(0,1)
# define inverse function
Finv=lambda u: 1/alpha*log(1/(1-u))
```

```
# apply inverse function to samples
v = array(list(map(Finv,u.rvs(nsamp))))
```

Now, we have the samples from the exponential distribution, but how do we know the method is correct with samples distributed accordingly? Fortunately, `scipy.stats` already has an exponential distribution, so we can check our work against the reference using a *probability plot* (i.e., also known as a *quantile-quantile plot*). The following code sets up the probability plot from `scipy.stats`.

```
%matplotlib inline

from matplotlib.pyplot import setp, subplots
fig,ax = subplots()
fig.set_size_inches((7,5))
_=scipy.stats.probplot(v,(1,),dist='expon',plot=ax)
line=ax.get_lines()[0]
_=setp(line,'color','k')
_=setp(line,'alpha',.1)
line=ax.get_lines()[1]
_=setp(line,'color','gray')
_=setp(line,'lw',3.0)
_=setp(ax.yaxis.get_label(),'fontsize',18)
_=setp(ax.xaxis.get_label(),'fontsize',18)
_=ax.set_title('Probability Plot',fontsize=18)
_=ax.grid()
fig.tight_layout()
fig.savefig('Sampling_Monte_Carlo_005.png')
```



```
fig,ax=subplots()
scipy.stats.probplot(v,(1,),dist='expon',plot=ax)
```



```
((array([1.00069315, 1.0016833 , 1.00268512, 1.00368795, 1.00469178,
1.00569662, 1.00670247, 1.00770934, 1.00871722, 1.00972611,
1.01073603, 1.01174696, 1.01275892, 1.01377191, 1.01478592,
1.01580096, 1.01681703, 1.01783414, 1.01885228, 1.01987145,
1.02089167, 1.02191293, 1.02293524, 1.02395859, 1.02498299,
1.02600844, 1.02703494, 1.02806249, 1.02909111, 1.03012078,
1.03115151, 1.03218331, 1.03321617, 1.03425011, 1.03528511,
1.03632118, 1.03735833, 1.03839655, 1.03943586, 1.04047624,
1.04151771, 1.04256027, 1.04360391, 1.04464864, 1.04569447,
1.04674139, 1.04778941, 1.04883852, 1.04988874, 1.05094006,
1.05199249, 1.05304603, 1.05410068, 1.05515644, 1.05621332,
1.05727132, 1.05833044, 1.05939068, 1.06045204, 1.06151454,
1.06257816, 1.06364292, 1.06470881, 1.06577583, 1.066844 ,
1.06791331, 1.06898377, 1.07005537, 1.07112812, 1.07220202,
1.07327708, 1.07435333 , 1.07543067, 1.07650921, 1.07758891,
1.07866978, 1.07975181, 1.08083502, 1.08191941, 1.08300497,
1.08409171, 1.08517963, 1.08626874, 1.08735904, 1.08845052,
1.0895432 , 1.09063708, 1.09173215, 1.09282842, 1.09392589,
1.09502457, 1.09612446, 1.09722556, 1.09832788, 1.09943141,
1.10053616, 1.10164213, 1.10274933, 1.10385775, 1.1049674 ,
1.10607829, 1.10719041, 1.10830377, 1.10941837, 1.11053421,
1.11165131, 1.11276965, 1.11388924, 1.11501008, 1.11613219,
1.11725555, 1.11838018, 1.11950608, 1.12063324, 1.12176168,
1.12289139, 1.12402238, 1.12515465, 1.1262882 , 1.12742304,
1.12855917, 1.12969659, 1.1308353 , 1.13197532, 1.13311663,
1.13425925, 1.13540318, 1.13654841, 1.13769496, 1.13884283,
1.13999201, 1.14114252, 1.14229435, 1.14344751, 1.144602 ,
1.14575782, 1.14691499, 1.14807349, 1.14923334, 1.15039453,
1.15155708, 1.15272097, 1.15388623, 1.15505284, 1.15622082,
1.15739016, 1.15856087, 1.15973295, 1.16090641, 1.16208124,
1.16325746, 1.16443507, 1.16561406, 1.16679444, 1.16797622,
1.1691594 , 1.17034397, 1.17152996, 1.17271735, 1.17390615,
1.17509637, 1.176288 , 1.17748106, 1.17867554, 1.17987146,
1.1810688 , 1.18226758, 1.1834678 , 1.18466945, 1.18587256,
1.18707711, 1.18828312, 1.18949059, 1.19069951, 1.1919099 ,
1.19312175, 1.19433507, 1.19554987, 1.19676614, 1.1979839 ,
1.19920314, 1.20042387, 1.20164609, 1.20286981, 1.20409503,
1.20532175, 1.20654997, 1.20777971, 1.20901096, 1.21024373,
1.21147802, 1.21271383, 1.21395118, 1.21519005, 1.21643047,
1.21767242, 1.21891592, 1.22016097, 1.22140757, 1.22265572,
1.22390544, 1.22515672, 1.22640956, 1.22766398, 1.22891998,
1.23017755, 1.23143671, 1.23269745, 1.23395978, 1.23522372,
1.23648925, 1.23775638, 1.23902512, 1.24029548, 1.24156745,
1.24284104, 1.24411625, 1.24539309, 1.24667156, 1.24795168,
1.24923343, 1.25051682, 1.25180187, 1.25308857, 1.25437693,
1.25566694, 1.25695863, 1.25825199, 1.25954702, 1.26084373,
1.26214212, 1.26344422 , 1.26474398, 1.26604745, 1.26735262,
1.2686595 , 1.26996809, 1.27127839, 1.27259041, 1.27390416,
1.27521963, 1.27653684, 1.27785578, 1.27917647, 1.2804989 ,
1.28182308, 1.28314902, 1.28447672, 1.28580619, 1.28713742,
1.28847043, 1.28980522, 1.29114179, 1.29248015, 1.29382031,
1.29516226, 1.29650602, 1.29785158, 1.29919896, 1.30054815,
1.30189917, 1.30325202, 1.3046067 , 1.30596321, 1.30732157,
1.30868178, 1.31004384, 1.31140776, 1.31277354, 1.31414118,
1.3155107 , 1.3168821 , 1.31825539, 1.31963056, 1.32100762,
1.32238658, 1.32376745, 1.32515023, 1.32653492, 1.32792153,
1.32931007, 1.33070054, 1.33209294, 1.33348729, 1.33488358,
1.33628182, 1.33768203, 1.33908419, 1.34048833, 1.34189444,
1.34330253, 1.3447126 , 1.34612467, 1.34753873, 1.34895479,
1.35037287, 1.35179295, 1.35321506, 1.35463919, 1.35606535,
1.35749355, 1.35892379, 1.36035608, 1.36179043, 1.36322683,
1.36466531, 1.36610585, 1.36754847, 1.36899318, 1.37043997,
1.37188887, 1.37333986, 1.37479296, 1.37624818, 1.37770552,
1.37916499, 1.38062658, 1.38209032, 1.38355621, 1.38502424,
1.38649444, 1.38796679, 1.38944132, 1.39091803, 1.39239692,
1.393878 , 1.39536128, 1.39684676, 1.39833445, 1.39982436,
1.40131649, 1.40281085, 1.40430745, 1.40580629, 1.40730738,
1.40881073, 1.41031634, 1.41182422, 1.41333438, 1.41484682,
1.41636155, 1.41787858, 1.41939792, 1.42091957, 1.42244353,
1.42396983, 1.42549845, 1.42702942, 1.42856273, 1.4300984 ,
1.43163643, 1.43317683, 1.43471961, 1.43626476, 1.43781232,
1.43936226, 1.44091462, 1.44246939, 1.44402658, 1.4455862 ,
1.44714825, 1.44871275, 1.4502797 , 1.45184911, 1.45342099,
1.45499534, 1.45657217, 1.4581515 , 1.45973332, 1.46131765,
1.46290449, 1.46449385, 1.46608575, 1.46768018, 1.46927716,
1.47087669, 1.47247879, 1.47408345, 1.4756907 , 1.47730053,
1.47891296, 1.480528 , 1.48214564, 1.48376591, 1.4853888 ,
1.48701434, 1.48864252, 1.49027336, 1.49190686, 1.49354303,
1.49518188, 1.49682343, 1.49846767, 1.50011462, 1.50176429,
1.50341669, 1.50507181, 1.50672969, 1.50839031, 1.5100537 ,
1.51171986, 1.5133888 , 1.51506054, 1.51673507, 1.51841241,
1.52009256, 1.52177555, 1.52346137, 1.52515004, 1.52684156,
1.52853596, 1.53023322, 1.53193338, 1.53363642, 1.53534238,
1.53705125, 1.53876304, 1.54047777, 1.54219545, 1.54391608,
```

```

1.5456396, 1.54736624, 1.5490958, 1.55082836, 1.55256392,
1.5543025, 1.5560441, 1.55778875, 1.55953645, 1.5612872,
1.56304102, 1.56479793, 1.56655793, 1.56832103, 1.57008725,
1.57185659, 1.57362906, 1.57540469, 1.57718347, 1.57896542,
1.58075055, 1.58253888, 1.58433041, 1.58612515, 1.58792312,
1.58972433, 1.59152879, 1.59333652, 1.59514751, 1.59696179,
1.59877937, 1.60060026, 1.60242447, 1.60425202, 1.60608291,
1.60791716, 1.60975478, 1.61159578, 1.61344018, 1.61528798,
1.61713921, 1.61899387, 1.62085198, 1.62271354, 1.62457858,
1.6264471, 1.62831912, 1.63019465, 1.63207371, 1.6339563,
1.63584244, 1.63773215, 1.63962544, 1.64152231, 1.64342279,
1.64532689, 1.64723463, 1.64914601, 1.65106105, 1.65297976,
1.65490216, 1.65682827, 1.65875809, 1.66069165, 1.66262895,
1.66457001, 1.66651484, 1.66846347, 1.6704159, 1.67237215,
1.67433223, 1.67629616, 1.67826396, 1.68023564, 1.68221121,
1.68419069, 1.6861741, 1.68816145, 1.69015276, 1.69214804,
1.69414732, 1.69615059, 1.69815789, 1.70016922, 1.70218461,
1.70420407, 1.70622761, 1.70825526, 1.71028703, 1.71232293,
1.71436299, 1.71640722, 1.71845563, 1.72050825, 1.72256509,
1.72462617, 1.72669151, 1.72876112, 1.73083503, 1.73291324,
1.73499579, 1.73708267, 1.73917393, 1.74126956, 1.7433696,
1.74547405, 1.74758295, 1.7496963, 1.75181412, 1.75393644,
1.75606328, 1.75819465, 1.76033057, 1.76247106, 1.76461614,
1.76676584, 1.76892016, 1.77107914, 1.77324279, 1.77541113,
1.77758418, 1.77976196, 1.7819445, 1.78413181, 1.78632392,
1.78852084, 1.7907226, 1.79292922, 1.79514071, 1.79735711,
1.79957843, 1.8018047, 1.80403594, 1.80627216, 1.8085134,
1.81075967, 1.813011, 1.8152674, 1.81752891, 1.81979555,
1.82206734, 1.8243443, 1.82662645, 1.82891383, 1.83120645,
1.83350433, 1.83580751, 1.83811601, 1.84042985, 1.84274905,
1.84507365, 1.84740366, 1.84973911, 1.85208004, 1.85442645,
1.85677838, 1.85913586, 1.86149891, 1.86386755, 1.86624182,
1.86862174, 1.87100734, 1.87339864, 1.87579567, 1.87819846,
1.88060705, 1.88302144, 1.88544168, 1.88786779, 1.8902998,
1.89273774, 1.89518164, 1.89763153, 1.90008743, 1.90254938,
1.9050174, 1.90749153, 1.9099718, 1.91245824, 1.91495087,
1.91744973, 1.91995485, 1.92246626, 1.924984, 1.92750809,
1.93003857, 1.93257547, 1.93511882, 1.93766865, 1.940225,
1.94278791, 1.9453574, 1.94793351, 1.95051627, 1.95310572,
1.95570189, 1.95830482, 1.96091455, 1.9635311, 1.96615452,
1.96878484, 1.97142209, 1.97406632, 1.97671755, 1.97937584,
1.98204121, 1.98471371, 1.98739336, 1.99008022, 1.99277431,
1.99547568, 1.99818437, 2.00090042, 2.00362386, 2.00635474,
2.0090931, 2.01183898, 2.01459242, 2.01735346, 2.02012214,
2.02289851, 2.02568262, 2.02847449, 2.03127418, 2.03408173,
2.03689719, 2.03972059, 2.04255199, 2.04539143, 2.04823896,
2.05109461, 2.05395845, 2.05683051, 2.05971084, 2.06259949,
2.06549651, 2.06840195, 2.07131585, 2.07423827, 2.07716925,
2.08010885, 2.08305712, 2.08601411, 2.08897986, 2.09195444,
2.09493789, 2.09793027, 2.10093163, 2.10394203, 2.10696151,
2.10999014, 2.11302797, 2.11607506, 2.11913146, 2.12219723,
2.12527243, 2.12835712, 2.13145134, 2.13455518, 2.13766868,
2.1407919, 2.1439249, 2.14706776, 2.15022052, 2.15338325,
2.15655602, 2.15973888, 2.16293191, 2.16613517, 2.16934872,
2.17257263, 2.17580697, 2.17905181, 2.1823072, 2.18557323,
2.18884996, 2.19213746, 2.19543581, 2.19874507, 2.20206532,
2.20539663, 2.20873908, 2.21209273, 2.21545767, 2.21883397,
2.22222171, 2.22562096, 2.22903181, 2.23245433, 2.23588861,
2.23933472, 2.24279274, 2.24626277, 2.24974488, 2.25323916,
2.25674569, 2.26026456, 2.26379585, 2.26733966, 2.27089608,
2.27446518, 2.27804707, 2.28164184, 2.28524957, 2.28887037,
2.29250433, 2.29615154, 2.2998121, 2.30348611, 2.30717367,
2.31087487, 2.31458983, 2.31831864, 2.3220614, 2.32581823,
2.32958922, 2.33337448, 2.33717413, 2.34098828, 2.34481702,
2.34866048, 2.35251877, 2.35639201, 2.3602803, 2.36418377,
2.36810254, 2.37203673, 2.37598645, 2.37995184, 2.38393301,
2.3879301, 2.39194323, 2.39597253, 2.40001813, 2.40408016,
2.40815876, 2.41225406, 2.41636621, 2.42049533, 2.42464157,
2.42880508, 2.432986, 2.43718446, 2.44140063, 2.44563465,
2.44988668, 2.45415686, 2.45844535, 2.46275231, 2.46707791,
2.47142229, 2.47578563, 2.4801681, 2.48456985, 2.48899107,
2.49343192, 2.49789258, 2.50237322, 2.50687403, 2.5113952,
2.51593689, 2.52049931, 2.52508263, 2.52968706, 2.53431279,
2.53896002, 2.54362894, 2.54831977, 2.5530327, 2.55776795,
2.56252573, 2.56730625, 2.57210974, 2.57693641, 2.58178649,
2.58666021, 2.5915578, 2.5964795, 2.60142553, 2.60639616,
2.61139161, 2.61641214, 2.621458, 2.62652946, 2.63162676,
2.63675019, 2.64189999, 2.64707646, 2.65227985, 2.65751047,
2.66276859, 2.6680545, 2.67336851, 2.6787109, 2.68408199,
2.68948208, 2.69491149, 2.70037054, 2.70585955, 2.71137887,
2.71692881, 2.72250972, 2.72812196, 2.73376588, 2.73944182,
2.74515017, 2.75089129, 2.75666556, 2.76247337, 2.76831511,
2.77419117, 2.78010196, 2.7860479, 2.79202941, 2.79804691,
2.80410083, 2.81019164, 2.81631976, 2.82248568, 2.82868984,
2.83493274, 2.84121486, 2.84753669, 2.85389875, 2.86030154,

```

```

2.86674558, 2.87323143, 2.87975961, 2.88633069, 2.89294524,
2.89960383, 2.90630705, 2.91305551, 2.91984983, 2.92669062,
2.93357853, 2.94051421, 2.94749833, 2.95453157, 2.96161463,
2.96874822, 2.97593306, 2.9831699 , 2.99045949, 2.99780261,
3.00520006, 3.01265263, 3.02016116, 3.02772649, 3.0353495 ,
3.04303106, 3.05077208, 3.0585735 , 3.06643626, 3.07436133,
3.08234971, 3.09040241, 3.0985205 , 3.10670502, 3.11495708,
3.12327781, 3.13166835, 3.14012989, 3.14866364, 3.15727084,
3.16595277, 3.17471074, 3.18354608, 3.19246019, 3.20145447,
3.21053038, 3.21968942, 3.22893313, 3.23826308, 3.2476809 ,
3.25718825, 3.26678687, 3.27647851, 3.286265 , 3.29614821,
3.30613008, 3.31621259, 3.32639779, 3.3366878 , 3.34708479,
3.35759102, 3.36820881, 3.37894054, 3.38978869, 3.40075582,
3.41184456, 3.42305764, 3.43439788, 3.44586819, 3.45747161,
3.46921124, 3.48109034, 3.49311224, 3.50528043, 3.51759851,
3.53007022, 3.54269945, 3.55549021, 3.56844669, 3.58157325,
3.59487442, 3.60835489, 3.62201957, 3.63587357, 3.64992219,
3.664171 , 3.67862578, 3.69329256, 3.70817766, 3.72328768,
3.73862953, 3.75421041, 3.77003791, 3.78611995, 3.80246486,
3.81908137, 3.83597866, 3.85316638, 3.87065469, 3.8884543 ,
3.90657648, 3.92503315, 3.94383688, 3.96300098, 3.98253953,
4.00246745, 4.02280059, 4.04355576, 4.06475085, 4.08640492,
4.10853829, 4.13117265, 4.15433121, 4.17803884, 4.20232219,
4.22720993, 4.25273291, 4.27892442, 4.30582041, 4.33345984,
4.36188498, 4.3911418 , 4.42128045, 4.45235573, 4.48442775,
4.51756257, 4.55183303, 4.58731977, 4.62411229, 4.66231039,
4.70202572, 4.74338383, 4.78652651, 4.83161473, 4.87883228,
4.9283903 , 4.98053296, 5.03554476, 5.09375995, 5.15557477,
5.2214639 , 5.29200257, 5.36789712, 5.45002822, 5.53951332,
5.63780008, 5.74681071, 5.8691752 , 6.00862804, 6.17072894,
6.36428806, 6.60452834, 6.92137102, 7.38783943, 8.27461475]],
array([5.48434175e-05, 4.66450591e-04, 6.60747790e-03, 6.80784299e-03,
7.98792685e-03, 8.57578746e-03, 9.70724009e-03, 9.84811574e-03,
1.01542481e-02, 1.06570685e-02, 1.12796657e-02, 1.18245640e-02,
1.27140336e-02, 1.40886554e-02, 1.59301775e-02, 1.60806677e-02,
1.65499723e-02, 1.79201442e-02, 2.08516095e-02, 2.26275737e-02,
2.37411942e-02, 2.46200794e-02, 2.59244625e-02, 2.59266333e-02,
2.61407823e-02, 2.63726703e-02, 2.69121511e-02, 2.69239653e-02,
2.82371860e-02, 2.87964967e-02, 3.09261502e-02, 3.10625453e-02,
3.22016829e-02, 3.25144645e-02, 3.31997481e-02, 3.33506424e-02,
3.36920750e-02, 3.44748531e-02, 3.56925056e-02, 3.67480916e-02,
3.80294882e-02, 3.82171325e-02, 3.96070764e-02, 4.08823507e-02,
4.12431093e-02, 4.32347336e-02, 4.50387150e-02, 4.50960293e-02,
4.54830670e-02, 4.68388659e-02, 4.75958271e-02, 4.91055711e-02,
5.08861154e-02, 5.29874557e-02, 5.36952281e-02, 5.39776133e-02,
5.50310699e-02, 5.58212760e-02, 5.62257763e-02, 5.76647442e-02,
5.96029598e-02, 5.96492469e-02, 6.17702760e-02, 6.27020342e-02,
6.44874450e-02, 6.45241689e-02, 6.57686791e-02, 6.70925555e-02,
6.86139733e-02, 6.99544604e-02, 7.07921695e-02, 7.16643427e-02,
7.21679637e-02, 7.25597839e-02, 7.35443285e-02, 7.40383832e-02,
7.42785728e-02, 7.66492385e-02, 7.70081030e-02, 7.86500756e-02,
7.93387561e-02, 8.09480734e-02, 8.09687866e-02, 8.51450419e-02,
8.64988014e-02, 8.76096871e-02, 9.13382091e-02, 9.27489580e-02,
9.52462352e-02, 9.63404709e-02, 9.69218765e-02, 9.83402062e-02,
1.02374556e-01, 1.02409870e-01, 1.04023836e-01, 1.09200735e-01,
1.10512674e-01, 1.10666213e-01, 1.10834348e-01, 1.13115248e-01,
1.15732485e-01, 1.16084384e-01, 1.17172499e-01, 1.18011872e-01,
1.18713213e-01, 1.19397055e-01, 1.19914398e-01, 1.20907898e-01,
1.23106340e-01, 1.24164090e-01, 1.26116515e-01, 1.26259599e-01,
1.26713941e-01, 1.27409405e-01, 1.27574083e-01, 1.28376389e-01,
1.29308922e-01, 1.29410676e-01, 1.29845800e-01, 1.29887883e-01,
1.31142340e-01, 1.33531503e-01, 1.36330393e-01, 1.37176327e-01,
1.37304010e-01, 1.39074880e-01, 1.39801489e-01, 1.40268214e-01,
1.40535591e-01, 1.42003568e-01, 1.42209262e-01, 1.42489080e-01,
1.43995487e-01, 1.45098357e-01, 1.45288121e-01, 1.53062946e-01,
1.55098838e-01, 1.55907936e-01, 1.57118625e-01, 1.61713324e-01,
1.61724552e-01, 1.62599455e-01, 1.64461182e-01, 1.65199146e-01,
1.67127366e-01, 1.67293716e-01, 1.67529563e-01, 1.70024920e-01,
1.72499596e-01, 1.73460177e-01, 1.75262860e-01, 1.76347278e-01,
1.77037189e-01, 1.78128897e-01, 1.79163926e-01, 1.80247318e-01,
1.82903770e-01, 1.82931598e-01, 1.83024804e-01, 1.84192803e-01,
1.85939064e-01, 1.86051017e-01, 1.90963026e-01, 1.92325149e-01,
1.94165494e-01, 1.95437219e-01, 1.95802593e-01, 1.96133127e-01,
1.97905458e-01, 2.01380198e-01, 2.01780389e-01, 2.03380956e-01,
2.03611698e-01, 2.04201340e-01, 2.05005945e-01, 2.05163204e-01,
2.06342954e-01, 2.07272447e-01, 2.08179315e-01, 2.08843077e-01,
2.09275212e-01, 2.09372835e-01, 2.10043200e-01, 2.11793482e-01,
2.12547750e-01, 2.13655745e-01, 2.13990147e-01, 2.15023943e-01,
2.15832543e-01, 2.15881770e-01, 2.16869741e-01, 2.17226708e-01,
2.17665525e-01, 2.18093330e-01, 2.20788520e-01, 2.22197357e-01,
2.23639564e-01, 2.23912299e-01, 2.24258684e-01, 2.25668502e-01,
2.25785768e-01, 2.28004563e-01, 2.28587438e-01, 2.30325404e-01,
2.34419737e-01, 2.35349155e-01, 2.36903477e-01, 2.37523129e-01,
2.38503221e-01, 2.39007804e-01, 2.40291766e-01, 2.40798669e-01,
2.42517849e-01, 2.43235106e-01, 2.48743540e-01, 2.49315834e-01,

```



```

2.50197471e-01, 2.50615546e-01, 2.53257685e-01, 2.53595475e-01,
2.53609797e-01, 2.60491846e-01, 2.62245548e-01, 2.62643289e-01,
2.63744855e-01, 2.65154366e-01, 2.65553724e-01, 2.67187848e-01,
2.67953889e-01, 2.68128495e-01, 2.68218572e-01, 2.69527348e-01,
2.71513317e-01, 2.72174856e-01, 2.73231525e-01, 2.76013072e-01,
2.76743213e-01, 2.78437841e-01, 2.84424253e-01, 2.84848384e-01,
2.85264346e-01, 2.85637782e-01, 2.86468624e-01, 2.87645703e-01,
2.88652820e-01, 2.89785907e-01, 2.91473205e-01, 2.93619481e-01,
2.93647579e-01, 2.93809041e-01, 2.94075880e-01, 2.96389027e-01,
2.98049530e-01, 2.99139534e-01, 2.99728063e-01, 3.00005824e-01,
3.00344169e-01, 3.01332057e-01, 3.03241270e-01, 3.04901688e-01,
3.06620548e-01, 3.06775280e-01, 3.09537141e-01, 3.10758706e-01,
3.11118273e-01, 3.11745786e-01, 3.18681432e-01, 3.24613567e-01,
3.25067476e-01, 3.27460474e-01, 3.31922621e-01, 3.34301469e-01,
3.35017043e-01, 3.36020917e-01, 3.36221944e-01, 3.36456898e-01,
3.39918056e-01, 3.39963428e-01, 3.40767507e-01, 3.40793379e-01,
3.42851315e-01, 3.43453460e-01, 3.44107179e-01, 3.44480052e-01,
3.45646851e-01, 3.47748279e-01, 3.48136079e-01, 3.48578977e-01,
3.50387870e-01, 3.50955526e-01, 3.52936967e-01, 3.54885625e-01,
3.55712795e-01, 3.55838221e-01, 3.57496306e-01, 3.57565526e-01,
3.59052211e-01, 3.61949375e-01, 3.63108093e-01, 3.65490186e-01,
3.68300640e-01, 3.80633853e-01, 3.82792669e-01, 3.83763760e-01,
3.86209620e-01, 3.87468656e-01, 3.87482236e-01, 3.89298747e-01,
3.89538705e-01, 3.91729278e-01, 3.92796008e-01, 3.93462416e-01,
3.95762436e-01, 3.95815176e-01, 3.96350796e-01, 3.96524220e-01,
3.97839162e-01, 3.98341809e-01, 3.99313972e-01, 4.02131867e-01,
4.03700085e-01, 4.03959117e-01, 4.05006248e-01, 4.06153750e-01,
4.06436077e-01, 4.06832224e-01, 4.06963808e-01, 4.07230805e-01,
4.08375773e-01, 4.10617777e-01, 4.13914778e-01, 4.14211574e-01,
4.15403169e-01, 4.15632176e-01, 4.16543063e-01, 4.17114186e-01,
4.19198280e-01, 4.21183381e-01, 4.21204040e-01, 4.23704432e-01,
4.24074087e-01, 4.24718920e-01, 4.24991885e-01, 4.25048699e-01,
4.27078142e-01, 4.30651440e-01, 4.33774429e-01, 4.36822595e-01,
4.42063851e-01, 4.42665001e-01, 4.44848441e-01, 4.46886839e-01,
4.48156925e-01, 4.49015728e-01, 4.50634609e-01, 4.51416432e-01,
4.53512637e-01, 4.54489564e-01, 4.56093045e-01, 4.56793211e-01,
4.57892249e-01, 4.59229176e-01, 4.60617801e-01, 4.63069890e-01,
4.64191135e-01, 4.65173948e-01, 4.65584510e-01, 4.65621623e-01,
4.66310094e-01, 4.66470199e-01, 4.67732412e-01, 4.72734175e-01,
4.73669513e-01, 4.76478161e-01, 4.77345059e-01, 4.80218666e-01,
4.81822583e-01, 4.83859801e-01, 4.83913965e-01, 4.88115918e-01,
4.88501769e-01, 4.89772213e-01, 4.92575758e-01, 4.92910331e-01,
4.93631296e-01, 4.94740772e-01, 4.95793960e-01, 4.95941656e-01,
4.96235305e-01, 4.98935638e-01, 5.00668185e-01, 5.01656634e-01,
5.04088891e-01, 5.08497634e-01, 5.09781498e-01, 5.11205785e-01,
5.11711017e-01, 5.16995422e-01, 5.17091164e-01, 5.17685337e-01,
5.17907706e-01, 5.19204447e-01, 5.20015869e-01, 5.20106221e-01,
5.23252244e-01, 5.24169356e-01, 5.24222424e-01, 5.24312117e-01,
5.30023727e-01, 5.32112567e-01, 5.34111017e-01, 5.34400474e-01,
5.35872980e-01, 5.39571707e-01, 5.39959109e-01, 5.41755916e-01,
5.42058040e-01, 5.42126886e-01, 5.45746636e-01, 5.47431451e-01,
5.48146989e-01, 5.52825145e-01, 5.53035593e-01, 5.55825484e-01,
5.57791988e-01, 5.58281148e-01, 5.58797505e-01, 5.63799811e-01,
5.65785380e-01, 5.71821607e-01, 5.76261562e-01, 5.76374980e-01,
5.77217421e-01, 5.77261458e-01, 5.78270020e-01, 5.80367233e-01,
5.83364616e-01, 5.84589791e-01, 5.85823676e-01, 5.87849485e-01,
5.91603621e-01, 5.99116916e-01, 5.99466207e-01, 6.00057420e-01,
6.01722886e-01, 6.02702238e-01, 6.04796857e-01, 6.06018400e-01,
6.06149617e-01, 6.07240666e-01, 6.07332239e-01, 6.07508353e-01,
6.08184615e-01, 6.09862813e-01, 6.11910988e-01, 6.12304686e-01,
6.12786391e-01, 6.13936679e-01, 6.14230096e-01, 6.15688722e-01,
6.15997621e-01, 6.16195695e-01, 6.18573867e-01, 6.20342516e-01,
6.21754754e-01, 6.22544558e-01, 6.25849775e-01, 6.29546866e-01,
6.29886401e-01, 6.31755274e-01, 6.32727489e-01, 6.34637057e-01,
6.37738144e-01, 6.37760023e-01, 6.38119221e-01, 6.39448063e-01,
6.40765020e-01, 6.44525074e-01, 6.46331770e-01, 6.48919394e-01,
6.50397394e-01, 6.50587976e-01, 6.51032491e-01, 6.54309569e-01,
6.54590372e-01, 6.61205304e-01, 6.66616294e-01, 6.66731449e-01,
6.67268667e-01, 6.68499465e-01, 6.69605452e-01, 6.72338163e-01,
6.73142234e-01, 6.74757100e-01, 6.75382777e-01, 6.75947759e-01,
6.76919943e-01, 6.80892453e-01, 6.84287383e-01, 6.85434801e-01,
6.86875047e-01, 6.87721583e-01, 6.87971790e-01, 6.91344751e-01,
6.91697498e-01, 6.95158238e-01, 6.98308461e-01, 6.99058584e-01,
7.01089371e-01, 7.06429869e-01, 7.07627667e-01, 7.08004812e-01,
7.09342329e-01, 7.11575813e-01, 7.11663059e-01, 7.14206377e-01,
7.15306785e-01, 7.20098637e-01, 7.22104252e-01, 7.22723603e-01,
7.23646648e-01, 7.23953721e-01, 7.25428026e-01, 7.39048771e-01,
7.40191770e-01, 7.40976599e-01, 7.43642365e-01, 7.47627402e-01,
7.47987327e-01, 7.49637513e-01, 7.58878119e-01, 7.59384725e-01,
7.59419707e-01, 7.59734970e-01, 7.63830907e-01, 7.66552951e-01,
7.67479397e-01, 7.68066784e-01, 7.70657378e-01, 7.76911127e-01,
7.79557478e-01, 7.81175274e-01, 7.82109955e-01, 7.84727651e-01,
7.86126291e-01, 7.86141991e-01, 7.87622120e-01, 7.91325259e-01,
7.94109632e-01, 8.00544432e-01, 8.02496458e-01, 8.03125642e-01,
8.06577787e-01, 8.06517088e-01, 8.07417227e-01, 8.10427321e-01

```

```

0.0007770e+01, 0.00027000e+01, 0.0742237e+01, 0.10477737e+01,
8.10845666e-01, 8.10891362e-01, 8.11974163e-01, 8.22616214e-01,
8.24843777e-01, 8.25313756e-01, 8.28958230e-01, 8.29614639e-01,
8.32361819e-01, 8.33370855e-01, 8.33606312e-01, 8.36381195e-01,
8.42690381e-01, 8.47471147e-01, 8.47750807e-01, 8.49001064e-01,
8.49541586e-01, 8.49680037e-01, 8.51844200e-01, 8.53419237e-01,
8.53527516e-01, 8.53708933e-01, 8.55477608e-01, 8.55922189e-01,
8.56224233e-01, 8.69814322e-01, 8.70051837e-01, 8.70724322e-01,
8.73197296e-01, 8.74406239e-01, 8.84472364e-01, 8.84505216e-01,
8.88624728e-01, 8.89113191e-01, 8.89432328e-01, 8.96488363e-01,
8.97731728e-01, 9.03922308e-01, 9.07082119e-01, 9.13889007e-01,
9.16444006e-01, 9.19647193e-01, 9.20435044e-01, 9.20627192e-01,
9.24427400e-01, 9.32715082e-01, 9.33008648e-01, 9.37331167e-01,
9.49838489e-01, 9.50259888e-01, 9.54566585e-01, 9.58769255e-01,
9.60518636e-01, 9.63406697e-01, 9.66736134e-01, 9.67153973e-01,
9.70495306e-01, 9.70857452e-01, 9.75839457e-01, 9.79113741e-01,
9.79870418e-01, 9.84431330e-01, 9.88701914e-01, 9.90255259e-01,
9.95292414e-01, 9.96350128e-01, 9.98346223e-01, 1.00343209e+00,
1.01892854e+00, 1.02353536e+00, 1.02519148e+00, 1.02533407e+00,
1.02882250e+00, 1.02889516e+00, 1.03313887e+00, 1.03352880e+00,
1.04167881e+00, 1.04274698e+00, 1.05316100e+00, 1.05342946e+00,
1.05989385e+00, 1.06821113e+00, 1.06964478e+00, 1.07491115e+00,
1.07627674e+00, 1.07758513e+00, 1.07773185e+00, 1.07787810e+00,
1.07906123e+00, 1.08019800e+00, 1.08046216e+00, 1.08101744e+00,
1.08917122e+00, 1.09136300e+00, 1.09332194e+00, 1.09517014e+00,
1.09702348e+00, 1.09885143e+00, 1.10105741e+00, 1.10207820e+00,
1.10321860e+00, 1.10546636e+00, 1.10658867e+00, 1.10679766e+00,
1.10704790e+00, 1.11131480e+00, 1.11360550e+00, 1.11725204e+00,
1.11841359e+00, 1.12101173e+00, 1.12290658e+00, 1.12299307e+00,
1.12470322e+00, 1.12804762e+00, 1.12816803e+00, 1.13001232e+00,
1.13495426e+00, 1.13836955e+00, 1.13983004e+00, 1.14047289e+00,
1.14346327e+00, 1.15284353e+00, 1.15479775e+00, 1.15776408e+00,
1.15786533e+00, 1.15908453e+00, 1.16273081e+00, 1.16522489e+00,
1.16740492e+00, 1.17829888e+00, 1.17850633e+00, 1.17902650e+00,
1.18183302e+00, 1.18864218e+00, 1.19148101e+00, 1.20005587e+00,
1.21324204e+00, 1.21817582e+00, 1.22068087e+00, 1.22872156e+00,
1.23110395e+00, 1.23246731e+00, 1.23389837e+00, 1.24217399e+00,
1.24217666e+00, 1.24861811e+00, 1.25018656e+00, 1.26058496e+00,
1.26465008e+00, 1.27050005e+00, 1.27133547e+00, 1.27502642e+00,
1.27908548e+00, 1.28030864e+00, 1.28042452e+00, 1.28109143e+00,
1.29063713e+00, 1.29148817e+00, 1.29536237e+00, 1.30338976e+00,
1.30627420e+00, 1.31168953e+00, 1.31361228e+00, 1.31857733e+00,
1.32304065e+00, 1.32449248e+00, 1.33886552e+00, 1.34939924e+00,
1.35211828e+00, 1.35490732e+00, 1.35617251e+00, 1.35660761e+00,
1.35961224e+00, 1.36817315e+00, 1.36861313e+00, 1.37025845e+00,
1.37530097e+00, 1.38012723e+00, 1.38300539e+00, 1.38447944e+00,
1.38980181e+00, 1.39014565e+00, 1.39482957e+00, 1.39979909e+00,
1.40035734e+00, 1.40946867e+00, 1.41042195e+00, 1.41356136e+00,
1.42293813e+00, 1.42731938e+00, 1.43102030e+00, 1.43181055e+00,
1.43834282e+00, 1.43864068e+00, 1.43914397e+00, 1.44454086e+00,
1.45100108e+00, 1.45473573e+00, 1.45884344e+00, 1.46201494e+00,
1.47759872e+00, 1.47771752e+00, 1.47792255e+00, 1.47927238e+00,
1.48712871e+00, 1.48801209e+00, 1.49838798e+00, 1.49959469e+00,
1.50128220e+00, 1.50539987e+00, 1.51085387e+00, 1.51183556e+00,
1.51294527e+00, 1.52029885e+00, 1.52112127e+00, 1.53155449e+00,
1.53619817e+00, 1.54109048e+00, 1.54655848e+00, 1.55158082e+00,
1.55787142e+00, 1.56133336e+00, 1.57430209e+00, 1.57578678e+00,
1.57848127e+00, 1.58609601e+00, 1.58911891e+00, 1.58952630e+00,
1.59820670e+00, 1.60135585e+00, 1.60273810e+00, 1.60392327e+00,
1.61449027e+00, 1.61506750e+00, 1.62429051e+00, 1.62520084e+00,
1.62891028e+00, 1.63170196e+00, 1.63308848e+00, 1.63426688e+00,
1.63706244e+00, 1.64677095e+00, 1.64809507e+00, 1.64851886e+00,
1.64973870e+00, 1.65092086e+00, 1.66287521e+00, 1.66527522e+00,
1.66675502e+00, 1.68929059e+00, 1.68978332e+00, 1.69955065e+00,
1.70488982e+00, 1.70543651e+00, 1.70644443e+00, 1.71015527e+00,
1.71392969e+00, 1.72275916e+00, 1.72344081e+00, 1.72613650e+00,
1.72619968e+00, 1.72771671e+00, 1.73710396e+00, 1.73908599e+00,
1.74857904e+00, 1.75588992e+00, 1.76187753e+00, 1.76240639e+00,
1.77147851e+00, 1.78635320e+00, 1.79009684e+00, 1.79068363e+00,
1.80297879e+00, 1.80462779e+00, 1.81475031e+00, 1.81507598e+00,
1.81616598e+00, 1.81676515e+00, 1.81899375e+00, 1.83594998e+00,
1.85094371e+00, 1.85733871e+00, 1.86175002e+00, 1.86328693e+00,
1.86491800e+00, 1.88006721e+00, 1.88315400e+00, 1.88694427e+00,
1.89492185e+00, 1.90150453e+00, 1.90492384e+00, 1.90896630e+00,
1.91791539e+00, 1.92398824e+00, 1.92546699e+00, 1.92599485e+00,
1.92706426e+00, 1.92719115e+00, 1.94562161e+00, 1.96302810e+00,
1.96361224e+00, 1.96598549e+00, 1.98275329e+00, 1.99342267e+00,
1.99853981e+00, 2.00809835e+00, 2.00998841e+00, 2.01557409e+00,
2.01891026e+00, 2.02644149e+00, 2.02778574e+00, 2.02829498e+00,
2.06625382e+00, 2.07269298e+00, 2.07716177e+00, 2.09071141e+00,
2.09655279e+00, 2.10023851e+00, 2.10054180e+00, 2.10363808e+00,
2.10488808e+00, 2.11692096e+00, 2.11987968e+00, 2.14018149e+00,
2.14550576e+00, 2.14668868e+00, 2.15008566e+00, 2.15847929e+00,
2.16886298e+00, 2.18581949e+00, 2.18637718e+00, 2.19464401e+00,
2.20656329e+00, 2.20851168e+00, 2.21392631e+00, 2.22700753e+00,

```

Note that we have $u_1, u_2 \sim U[a, b]$ so that $f(x) = \frac{1}{b-a} \frac{f(u_1)}{M}$. The more the samples line match the diagonal line, the more they match the reference distribution (i.e. exponential distribution in this case). You may also want to try `(dist='norm')` in the code above to see what happens when the normal distribution is the reference distribution. The samples created using the inverse CDF method match the exponential reference distribution.



Rejection Method

In some cases, inverting the CDF may be impossible. The rejection method can handle this situation. The idea is to pick two uniform random variables $u_1, u_2 \sim U[a, b]$ so that

$$f(x) = \frac{1}{b-a} \frac{f(u_1)}{M} \approx \frac{\Delta x}{b-a} \frac{f(u_1)}{M}$$

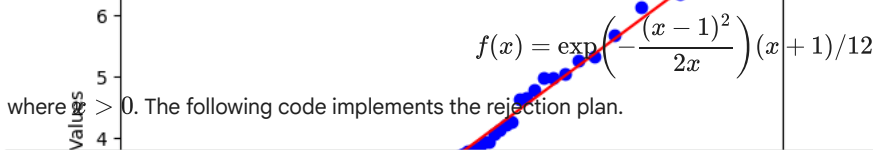
where we take $x = \frac{u_1 + u_2}{2}$. This is a two-stage process. First, draw u_1 uniformly from the interval $[a, b]$. Second, feed it into $f(x)$ and if $u_2 \leq f(x)$, then you have a valid sample for $f(x)$. Thus, u_1 is the proposed sample from f that may or may not be rejected depending on u_2 . The only job of the M constant is to scale down the $f(x)$ so that the u_2 variable can span the range. The efficiency of this method is the probability of accepting u_1 which comes from integrating out the above approximation,

$$\frac{\int_a^b f(x) dx}{M(b-a)} = \frac{1}{M(b-a)} \int_a^b f(x) dx = \frac{1}{M(b-a)}$$

This means that we don't want an unnecessarily large M because that makes it more likely that samples will be discarded.

Probability Plot

Let's try this method for a density that does not have a continuous inverse [^normalization]. [^normalization]: Note that this example density does not exactly integrate out to one like a probability density function should, but the normalization constant for this is distracting for our purposes here.



where $x > 0$. The following code implements the rejection plan.

```
import numpy as np
x = np.linspace(0.001,15,100)
f = lambda x: np.exp(-(x-1)**2/(2.*x))*(x+1)/12.
fx = f(x)
M=0.3 # scale factor
u1 = np.random.rand(10000)*15 # uniform random samples scaled out
u2 = np.random.rand(10000) # uniform random samples
idx, = np.where(u2<=f(u1)/M) # rejection criterion
v = u1[idx]
```

```
fig,ax=subplots()
fig.set_size_inches((9,5))
_ = ax.hist(v,density=True,bins=40,alpha=.3,color='gray')
_ = ax.plot(x,fx,'k',lw=3.,label='$f(x)$')
_ = ax.set_title('Estimated Efficiency=%3.1f%%'%(100*len(v)/len(u1)),
                fontsize=18)
_ = ax.legend(fontsize=18)
_ = ax.set_xlabel('$x$', fontsize=24)
fig.tight_layout()
fig.savefig('Sampling_Monte_Carlo_007.png')
```